Kubernetes Master Training Guide

From containers to enterprise hybrid clusters

ShadowXLab · Premium Enterprise Edition · Linux + Windows

Format: PDF · ~55 pages
Audience: Beginner → Engineer → Enterprise
Author: ShadowXLab Curriculum Team

© ShadowXLab · Enterprise Kubernetes Academy

Chapter 1 — Why Kubernetes Exists

Before Kubernetes, deploying applications meant manually provisioning VMs, configuring OS dependencies, and hoping every server stayed identical. **Containers** standardised the unit of deployment; **Kubernetes** standardised the way we run thousands of them.

The three problems Kubernetes solves

1. Scheduling. Given 100 servers and 500 workloads, which workload runs where? Kubernetes places pods on nodes that have capacity and match their constraints.

2. Self-healing. When a node crashes, replicas are rescheduled automatically. When a container exits, it is restarted under policy.

3. Declarative state. You describe *what* you want (3 replicas of nginx, accessible on port 80). Kubernetes continually reconciles the cluster to match.

Enterprise impact: A Fortune-500 bank moved from 4-week release cycles on VMs to 40-times-per-day deployments on Kubernetes — same SREs, same uptime SLOs.

Chapter 2 — Cluster Architecture

A Kubernetes cluster has two planes: the **Control Plane** (the brain) and the **Worker Nodes** (the muscles).

Control plane components

Component	Role	Failure impact
kube-apiserver	REST front door; all reads/writes go through it	Cluster becomes read-only; no scheduling
etcd	Distributed key-value store; the cluster's source of truth	Cluster state lost; restore from backup
kube-scheduler	Decides which node a new pod runs on	New pods stuck in Pending
kube-controller-manager	Reconciles desired vs actual (replicas, endpoints, nodes)	Drift; deleted pods not replaced; endpoints stale
cloud-controller-manager	Talks to cloud APIs (LBs, volumes, routes)	New LoadBalancers and PV provisioning broken

Worker node components

Component	Role	Failure impact
kubelet	Agent that runs pods on the node	Node goes NotReady; pods evicted
kube-proxy	Programs iptables/IPVS for Service routing	Service IPs stop routing
Container runtime	containerd / CRI-O — actually runs containers	Existing pods may run; new ones won't start
CNI plugin	Provides pod networking (Calico, Cilium, Flannel)	Pods get no IP; cross-node traffic fails

Linux vs Windows: The control plane is Linux-only. Windows joins as **worker nodes** running kubelet.exe, kube-proxy.exe and the HNS (Host Networking Service) instead of iptables.

Chapter 3 — The Pod Lifecycle

A Pod moves through six phases. Knowing them is the single biggest troubleshooting lever you have.

Phase	Meaning	Common cause if stuck
Pending	Accepted by API server, not yet scheduled	Scheduler can't place (resources, taints)
ContainerCreating	Image pulled, volumes mounted, starting	ImagePullBackOff, CNI failure, volume issue
Running	All containers started, at least one healthy	(healthy)

Succeeded	All containers exited with code 0	Batch / Job complete
Failed	At least one container exited non-zero	App crash, OOMKilled
CrashLoopBackOff	Container keeps restarting	Bad config, missing env, failing liveness probe

Probes

Startup probe — gives slow apps time to boot before liveness kicks in.
Liveness probe — restarts the container if the app is wedged.
Readiness probe — removes the pod from Service endpoints until ready.

Chapter 4 — Networking Internals

Every pod gets its own IP. Containers in a pod share that IP via a single Linux network namespace (the *pause* container).

Service types — when to use which

Type	Use case	How it works
ClusterIP	In-cluster east-west	Virtual IP, kube-proxy load-balances to pods
NodePort	Quick external access in dev	Opens 30000–32767 on every node
LoadBalancer	Production external traffic	Cloud LB → NodePort → Service → Pods
ExternalName	Alias to external DNS	CNAME at the DNS level, no proxy

Ingress vs Service

A Service exposes a single workload. An **Ingress** routes by hostname/path across many Services through one external IP. NGINX, Traefik and AWS ALB are common controllers.

Request flow

```
User → DNS → Ingress LB → Ingress controller pod → Service → kube-proxy → Pod IP → container
```

Chapter 5 — Storage

Volumes live as long as the pod. **PersistentVolumes (PV)** outlive pods and are claimed via **PersistentVolumeClaims (PVC)**.

StorageClass — dynamic provisioning

A StorageClass tells Kubernetes how to create a PV on demand (EBS, Azure Disk, GCE PD, Ceph, Longhorn). The PVC asks for size + class; the controller does the rest.

Why PVC Pending

1. No StorageClass marked default. 2. CSI driver crashed. 3. Cloud quota exhausted. 4. Access mode mismatch (RWX on a RWO-only backend).

Chapter 6 — Security & RBAC

RBAC = Roles + RoleBindings. A Role grants verbs (get, list, create) on resources within a namespace; a ClusterRole works cluster-wide.

```
kubectl create role pod-reader --verb=get,list --resource=pods
kubectl create rolebinding alice-pods --role=pod-reader --user=alice
```

Other layers

NetworkPolicy — pod-level firewall (requires CNI support).
PodSecurityAdmission — replaces PSP; enforces restricted/baseline/privileged.
Secrets — base64 by default; enable etcd encryption-at-rest in production.

Chapter 7 — Linux ↔ Windows Hybrid Clusters

Modern enterprises mix .NET Framework workloads (Windows) with cloud-native services (Linux) in **one** cluster. Control plane is always Linux.

Concern	Linux node	Windows node
Kubelet	kubelet (systemd unit)	kubelet.exe (Windows service)
Runtime	containerd / CRI-O	containerd with Windows shim
Networking	iptables / IPVS, CNI	HNS (Host Networking Service), win-overlay
Isolation	Linux namespaces + cgroups	Process or Hyper-V isolation
Image base	alpine, distroless, ubuntu	mcr.microsoft.com/windows/nanoserver, servercore
Logs path	/var/log/pods/	C:\var\log\pods\

Scheduling rule: Windows pods must carry nodeSelector: kubernetes.io/os: windows and tolerate the os=windows:NoSchedule taint, otherwise the scheduler may try a Linux node.

Chapter 8 — Observability Stack

Metrics: Prometheus scrapes /metrics endpoints; Grafana visualises.
Logs: Loki or ELK aggregates container stdout/stderr.
Traces: OpenTelemetry collector → Jaeger / Tempo.
Events: kubectl get events --sort-by=.lastTimestamp — the single most under-used troubleshooting command.

Chapter 9 — Troubleshooting Methodology

Follow the layers from outside in: **DNS** → **Ingress** → **Service** → **Endpoints** → **Pod** → **Container** → **Node** → **Control plane**. Stop at the first layer that's wrong.

The four kubectl commands you will use every day

```
kubectl get pods -A -o wide
kubectl describe pod <name>
kubectl logs <name> --previous
kubectl get events --sort-by=.lastTimestamp
```

Chapter 10 — Enterprise Patterns

GitOps (ArgoCD, Flux) — Git is the only way to change the cluster.
Multi-tenancy — one cluster per business unit vs namespace-per-tenant + quotas + NetworkPolicies.
Cost control — Kubecost / OpenCost tag spend per namespace, label or team.
DR — Velero backs up etcd + PVs; restore in another region in <30 min.